Git & GitHub Cheatsheet

Cheatsheet for DevOps Engineers

Installation & Account Setup :

Git Installation: Use this Blog for Git Installation on different OS

Setting Up a GitHub Account : Read this blog for creating a Github Account

Git Commands:

Repository Management:

Command	Description
git init	Initialize a new repository
git clone <repo>	Clone an existing repository
git remote -v	View remote repository details
git remote add origin <url>	Link local repo to remote

Adding and Committing:

Command	Description
git add <file>	Stage changes for commit
git add .	Stage all changes
git commit -m "message"	Commit changes with a message
git commit --amend	Edit the last commit

Branching and Merging:

Command	Description
git branch	List branches
git branch <branch>	Create a new branch
git checkout -b <branch>	Create and switch to a new branch
git merge <branch>	Merge a branch into current branch
git branch -d <branch>	Delete a branch
git checkout <branch>	Switch to a branch

Log and History:

Command	Description
git log	View commit history
git log --oneline	View commit history in one line
git diff	View unstaged changes
git diff HEAD	Compare working directory to HEAD

Undo Changes:

Command	Description
git reset <file>	Unstages a file. Changes remain in the working directory.
git restore <file>	Restores a file to its last committed state.
git reset --hard <commit>	Resets the current branch to the specified commit and discards all local changes.
git revert <commit>	Create a new commit that undoes a previous commit

Pushing and Pulling:

Command	Description
git push origin <branch>	Push changes to the remote repository
git pull origin <branch>	Pull changes from the remote repository

Log Git Config Commands:

Command	Description
git config --global user.name "<name>"	Sets the global username for Git commits.
git config --global user.email "<email>"	Sets the global email for Git commits.
git config --list	Displays the current Git configuration (user details, editor, etc.).

Git Status Command:

Command	Description
git status	Shows the current status of the working directory and staging area. Indicates tracked, modified, untracked, or staged files.

Git Fetch Command:

Command	Description
git fetch	Downloads commits, files, and references from a remote repository without merging them into the local branch.

Advanced Commands

Stashing:

Command	Description
git stash	*Save changes to a stash*
git stash list	List all stashes
git stash apply	Apply the last stash
git stash drop	Delete the last stash

Rebase and Cherry-pick:

Command	Description
git rebase <branch_name>	*Reapplies commits on top of another base branch. It rewrites history, so avoid using it on shared branches.*
git rebase -i <commit_hash>	*Allows you to rebase interactively from a specific commit.*
git rebase --abort	*Aborts the rebase process and returns to the state before starting the rebase.*
git rebase --continue	*Continues the rebase after resolving conflicts.*
git rebase --skip	*Skips the current commit during rebase.*
git cherry-pick <commit_hash>	*Applies a specific commit from another branch to the current branch.*

Tagging:

Command	Description
git tag <tag>	*Create a lightweight tag*
git tag -a <tag> -m "msg"	Create an annotated tag
git push origin --tags	Push tags to remote

Git Bisect Command (Debugging to Find Bad Commits):

Command	Description
git bisect start	*Begins the binary search to find the commit that introduced a bug.*
git bisect bad	*Marks the current commit as bad (contains the bug).*
git bisect good <commit>	*Marks a known good commit (does not contain the bug) to start the bisecting process.*
git bisect reset	*Ends the bisect process and resets the repository to the original state.*

Collaboration Commands:

Command	Description
git blame <file>	Shows who modified each line of the file
git shortlog	Summarizes contributions by author
git log --graph	Visualize branch history
git show <commit>	Displays details of a specific commit
git diff --staged	Show differences between staged changes and the last commit

GitHub Push Methods

1. SSH Method

1. Generate an SSH key:
using : `ssh-keygen`

2. Copy the SSH key:
using : ` cat ~/.ssh/<key_name>.pub`

3. Navigate to GitHub > Settings > SSH and GPG keys > New SSH key.

4. Paste the key and save.

5. Verify the SSH connection:
using : `ssh -T git@github.com`

6. Now clone the repo using ssh and do whatever you want and add & commit the changes

7. Push code:
using : ` git push origin main`

2. HTTPS Method

1. Firstly Generate a Personal Access Token

2. Use HTTPS URL for your repository:
`git remote set-url origin https://<Your_PAT>@github.com/<username>/<repository>.git`

3. Add and commit the changes

4. Push code:
`git push origin main/master`

Helpful GitHub Tips :

Create Pull Request :

1. Push <Your_branch> branch to GitHub.
2. Open the repository on GitHub.
3. Go to the "Pull Requests" tab and click "New Pull Request."
4. Select branches and create PR.

GitHub Fork and Sync Workflow :

1. Fork a Repository:
a. Go to the repository on GitHub and click on Fork.

2. Clone the Fork:
a. `git clone https://github.com/your-username/repo-name.git`

3. Sync Changes from Upstream:

a. git remote add upstream https://github.com/original-owner/repo-name.git
b. git fetch upstream
c. git merge upstream/main

Git Aliases :

Speed up your workflow by creating aliases:

1. *git config --global alias.co checkout*
2. *git config --global alias.br branch*
3. *git config --global alias.ci commit*

GitHub Actions:

- **Significance:** *Automates workflows like testing, building, and deploying code.*
- **Example:**

```
name: CI/CD Pipeline          # Name of the workflow
on:
  push:          # Trigger workflow on push to the main branch
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  build:          # Define a job named 'build'
    runs-on: ubuntu-latest                # Use the latest Ubuntu environment
    steps:
      - name: Checkout Code
        uses: actions/checkout@v2
      - name: Install Dependencies
        run: npm install
      - name: Run Tests
        run: npm test
```

Description: Runs tests on every push or pull request to the main branch.

References:

- *Git Documentation*
- *GitHub Docs*